

Statistical Programming II

Practical 4: Version Control with Git and GitHub

中德双语复习笔记

Sommersemester 2026

目录

第一部分 中文详解	3
1 本次 Practical 的目标	3
2 Git 的核心模型	3
3 Exercise 0: 检查 Git 设置和仓库状态	4
3.1 检查用户名和邮箱	4
3.2 进入 Practical 3 文件夹	4
3.3 检查 commit 历史和远程仓库	4
3.4 判断当前仓库状态	5
4 Exercise 1: 把本地仓库连接到 GitHub	5
4.1 在 GitHub 创建空仓库	5
4.2 SSH 身份验证	6
4.3 连接本地仓库和 GitHub	6
5 Exercise 2: 使用 .gitignore	7
5.1 .gitignore 的作用	7
5.2 常见匹配规则	8
5.3 为什么文件仍然存在?	8
6 Exercise 3: 查看和恢复历史版本	8
6.1 查看历史	8
6.2 故意制造错误	9
6.3 恢复单个文件	9
6.4 checkout、restore 和 revert	9

7 Exercise 4: Clone, Edit, Commit, Push	10
7.1 克隆仓库	10
8 Exercise 5: fetch 和 pull	11
8.1 git fetch	11
8.2 git pull	11
9 Exercise 6: Reflection Log	12
10 小组项目中的推荐工作流	13
11 命令速查表	13
 第二部分 Deutsche Zusammenfassung	 15
12 Lernziele	15
13 Konfiguration und Diagnose	15
14 Lokales Repository mit GitHub verbinden	15
15 .gitignore	16
16 Historie prüfen und Dateien wiederherstellen	16
17 Repository klonen und synchronisieren	17
18 fetch und pull	17
19 Empfohlener Workflow für das Gruppenprojekt	17

第一部分 中文详解

1 本次 Practical 的目标

Practical 4 的重点不是孤立地背诵 Git 命令，而是完整练习一次真实的版本控制流程：

创建本地仓库 → 建立 commit 历史 → 连接 GitHub → push / pull → 检查并恢复旧版本

完成后，应当能够回答以下问题：

- 文件保存、`git add`、`git commit` 和 `git push` 分别做了什么？
- 本地仓库与 GitHub 远程仓库之间是什么关系？
- 为什么需要 `.gitignore`？
- 如何查看历史并恢复某一个文件？
- `git fetch` 和 `git pull` 有什么区别？

2 Git 的核心模型

Git 工作流中可以区分四个位置：

位置	含义	相关操作或命令
Working Directory	当前正在编辑的文件	编辑并保存文件
Staging Area	为下一次 commit 准备好的修改	<code>git add</code>
Local Repository	电脑中保存的 commit 历史	<code>git commit</code> 、 <code>git log</code>
Remote Repository	GitHub 等平台上的远程仓库	<code>git push</code> 、 <code>git fetch</code> 、 <code>git pull</code>

表 1: Git 中的四个主要位置

修改文件 $\xrightarrow{\text{git add}}$ 暂存区 $\xrightarrow{\text{git commit}}$ 本地历史 $\xrightarrow{\text{git push}}$ GitHub

最容易混淆的地方

`git add` 不会创建版本，`git commit` 也不会自动上传 GitHub。

- `git add`：选择哪些修改进入下一次 commit。
- `git commit`：在本地创建一个正式版本。
- `git push`：把本地已有的 commit 上传到远程仓库。

3 Exercise 0: 检查 Git 设置和仓库状态

3.1 检查用户名和邮箱

```
git config --global user.name
git config --global user.email
```

这些信息会记录在 commit 中，用于说明是谁创建了该版本。若尚未设置，可以使用：

```
git config --global user.name "Youpeng Jiang"
git config --global user.email "you@example.com"
```

--global 表示该设置对当前电脑上的所有 Git 仓库生效。邮箱最好使用 GitHub 账户中已经验证的邮箱，或 GitHub 提供的 noreply 邮箱。即使邮箱未验证，commit 本身依然有效，但 GitHub 可能不会把它关联到个人贡献记录。

3.2 进入 Practical 3 文件夹

```
cd path/to/your/practical3-folder
```

Windows Git Bash 示例：

```
cd "/c/Data/StatProg2/practical_3"
```

如果路径中有空格，应当使用引号包住完整路径。

3.3 检查 commit 历史和远程仓库

```
git log --oneline
git remote -v
git status
```

- `git log --oneline`: 用简洁格式查看 commit 历史。
- `git remote -v`: 查看本地仓库连接了哪些远程地址。
- `git status`: 查看当前分支、修改、暂存和未跟踪文件。

`git log` 的输出可能类似：

```
8bd527a fix factorial function
34dc819 add debugging exercise
```

左边是 commit hash 的缩写，右边是 commit message。如果终端进入分页显示，按 `q` 即可退出。

`git remote -v` 的输出可能类似：

```
origin  git@github.com:username/statprog-debugging.git (fetch)
origin  git@github.com:username/statprog-debugging.git (push)
```

`origin` 是远程仓库的默认名称。它是惯例，而不是不可更改的固定关键词。

3.4 判断当前仓库状态

<code>git log</code>	<code>git remote -v</code>	说明
报错或没有 <code>commit</code>	没有输出	尚未建立正常的本地仓库
有 <code>commit</code>	没有输出	有本地仓库，但尚未连接 GitHub
有 <code>commit</code>	有 GitHub URL	本地仓库已经连接远程仓库

表 2: 根据 Git 输出判断仓库状态

如果当前文件夹还不是 Git 仓库，可以运行：

```
git init
git add .
git commit -m "add debugging exercise"
```

- `git init`: 在当前文件夹中创建 `.git`，使其成为 Git 仓库。
- `git add .`: 暂存当前目录中所有未被忽略的修改。
- `git commit -m "..."`: 为这些修改创建第一个本地版本。

不要随便删除 `.git`

删除 `.git` 会彻底删除该仓库的本地 `commit` 历史、分支、`remote` 配置、`stash` 和其他 Git 记录。只有当练习仓库的历史完全不需要保留，并且工作文件已经备份时，才可以考虑删除后重建。正式项目和小组项目中不能把它当作常规修复方法。

4 Exercise 1: 把本地仓库连接到 GitHub

4.1 在 GitHub 创建空仓库

如果本地仓库已经包含 `commit`，那么在 GitHub 创建新仓库时最好：

- 输入仓库名称，例如 `statprog-debugging`；
- 按课程要求选择 `Public` 或 `Private`；
- 不要让 GitHub 自动创建 `README`、`license` 或 `.gitignore`。

原因是：如果 GitHub 自动创建 `README`，远程仓库会产生一个本地没有的 `commit`。此时本地历史和远程历史从不同起点开始，第一次 `push` 可能会被拒绝。这个问题可以解决，但对初学者而言，创建空仓库更简单。

4.2 SSH 身份验证

SSH 使用一对密钥：

密钥	保存位置	能否公开
Private key 私钥	只保存在自己的电脑上	绝对不能
Public key 公钥	添加到 GitHub 账户	可以

表 3: SSH 密钥对

配置完成后，可以测试连接：

```
ssh -T git@github.com
```

成功时会看到类似信息：

```
Hi YOUR-USERNAME! You've successfully authenticated...
```

该命令只测试身份，不会修改或上传仓库。如果出现 `Permission denied (publickey)`，常见原因包括：

- 公钥没有正确添加到 GitHub；
- 添加了错误的文件，而不是以 `.pub` 结尾的公钥；
- 私钥没有被 SSH agent 加载；
- 仓库 remote 使用的是 HTTPS URL，而不是 SSH URL。

4.3 连接本地仓库和 GitHub

```
git remote add origin git@github.com:YOUR-USERNAME/statprog-debugging.git
git branch -M main
git push -u origin main
```

`git remote add origin ...` 把给定的 GitHub 地址注册为名叫 `origin` 的远程仓库。这一步只保存地址，不会上传任何文件。

`git branch -M main` 把当前分支重命名为 `main`。`-M` 表示强制重命名。

`git push -u origin main` 把本地 `main` 分支上传到远程 `origin`。`-u` 会建立 `upstream` 关系：

`main → origin/main`

此后通常只需运行：

```
git push
git pull
```

5 Exercise 2: 使用 .gitignore

5.1 .gitignore 的作用

.gitignore 告诉 Git:

这些文件可以存在于电脑中，但不应该纳入版本控制。

适合 R 和 Quarto 项目的示例:

```
# R session artefacts
.Rhistory
.RData
.Rproj.user/

# Quarto build output
/_site/
/.quarto/
*_files/

# Operating-system files
.DS_Store
Thumbs.db
```

文件或文件夹	为什么通常要忽略
.Rhistory	R Console 历史，不是可复现的分析代码
.RData	自动保存整个 Workspace，内容不透明，不利于复现
.Rproj.user/	当前电脑上的 RStudio 个人设置
.quarto/	Quarto 生成的临时构建内容
_site/	默认的网站生成结果
.DS_Store、 Thumbs.db	操作系统自动产生的无关文件

表 4: R/Quarto 项目中常见的忽略对象

项目规则优先

如果小组项目通过 docs/ 发布 GitHub Pages，就不能把 docs/ 随意加入 .gitignore。同样，是否忽略 data/ 取决于课程要求、数据大小和版权条件，不能机械照抄模板。

5.2 常见匹配规则

规则	含义
.RData	忽略名为 .RData 的文件
*.csv	忽略所有 CSV 文件
data/	忽略整个 data 文件夹
!data/README.md	即使忽略 data/，仍保留该 README
/_site/	只忽略仓库根目录下的 _site

表 5: .gitignore 常见规则

编辑完成后提交：

```
git add .gitignore
git commit -m "add gitignore for R and Quarto"
git push
```

5.3 为什么文件仍然存在？

.gitignore 不会删除电脑上的文件，只会阻止未跟踪文件进入 Git。如果一个文件以前已经被 commit，那么新增忽略规则不会让 Git 自动停止跟踪它。

对于单个文件：

```
git rm --cached path/to/file
git commit -m "stop tracking file"
git push
```

对于整个文件夹：

```
git rm -r --cached data/
```

--cached 表示只从 Git 的索引中移除，电脑上的本地文件仍会保留。

秘密信息不能只靠删除文件解决

如果 API key、密码或 token 已经被 commit，后来删除当前文件并不能确保秘密消失，因为旧 commit 仍保存它。应立即撤销并重新生成该密钥；必要时还要清理 Git 历史。

6 Exercise 3: 查看和恢复历史版本

6.1 查看历史

```
git log --oneline
git log --oneline --graph
```



```
git log -- practical3.qmd
```

第三条命令中的 `--` 用来区分 Git 参数和文件路径。它表示：

只查看与 `practical3.qmd` 有关的 commit。

查看某个 commit 的具体改动：

```
git show <commit-hash>
```

查看某个文件在指定 commit 中的完整内容：

```
git show <commit-hash>:practical3.qmd
```

冒号左边指定 commit，右边指定该 commit 中的文件路径。

6.2 故意制造错误

题目要求删除或改坏 `.qmd` 文件的一部分，然后提交：

```
git add .  
git commit -m "deliberately break something for exercise 3"
```

这样是在模拟一种现实情况：

错误修改已经被 commit，现在需要从历史中找回正确版本。

6.3 恢复单个文件

假设错误发生前的 commit hash 是 `34dc819`：

```
git checkout 34dc819 -- practical3.qmd
```

它不会把整个仓库切换到旧版本，而是从 `34dc819` 中取出 `practical3.qmd`，覆盖当前文件。其他文件不受影响。

随后提交恢复结果：

```
git commit -m "restore practical3.qmd to pre-break version"
```

现代 Git 中也可以使用语义更清楚的写法：

```
git restore --source=34dc819 --staged --worktree practical3.qmd
```

6.4 checkout、restore 和 revert

在已经共享的仓库中，`git revert` 通常比改写历史更安全，因为它保留旧 commit，并通过一个新 commit 明确记录撤销操作。

命令	作用
<code>git checkout <hash></code> <code>-- file</code>	从旧 commit 取回一个文件
<code>git restore</code> <code>--source=<hash> file</code>	用含义更明确的新命令恢复文件
<code>git revert <hash></code>	创建一个新的 commit，用来撤销指定旧 commit 的修改

表 6: 三个恢复相关命令的区别

7 Exercise 4: Clone、Edit、Commit、Push

7.1 克隆仓库

```
git clone git@github.com:YOUR-USERNAME/statprog-debugging.git statprog-copy
cd statprog-copy
```

`git clone` 不只是下载当前文件，它会：

1. 下载 GitHub 仓库；
2. 创建 `statprog-copy` 文件夹；
3. 建立本地 `.git`；
4. 下载 commit 历史；
5. 自动设置 `origin`；
6. 检出默认分支。

在副本中修改文件后：

```
git add .
git commit -m "add comment from practical 4"
git push
```

回到原始文件夹并获取修改：

```
cd ..
cd 03-debugging
git pull
```

该练习用两个本地文件夹模拟两台电脑或两个协作者：

副本 / 协作者 A $\xrightarrow{\text{push}}$ GitHub $\xrightarrow{\text{pull}}$ 原文件夹 / 协作者 B

Pull 前先看状态

如果原始文件夹中存在未 commit 的修改，并且与远程修改冲突，`git pull` 可能被拒绝或产生 merge conflict。因此在 pull 前先运行：`git status`。

8 Exercise 5: fetch 和 pull

8.1 git fetch

```
git fetch origin
```

它会下载 GitHub 上的新 commit，并更新 `origin/main`，但：

- 不修改当前打开的本地文件；
- 不自动合并到本地 `main`。

fetch 后可以把状态理解为：

- `main`：本地当前状态；
- `origin/main`：GitHub 最新状态的本地记录。

检查远程比本地多出的 commit：

```
git log HEAD..origin/main --oneline
```

查看本地和远程的具体差异：

```
git diff HEAD origin/main
```

确认后手动合并：

```
git merge origin/main
```

8.2 git pull

```
git pull
```

在课程采用的常见模型中，可以把它理解为：

```
git fetch  
git merge
```

也就是先下载远程 commit，再立即整合进当前分支。严格来说，具体整合方式也可能根据配置使用 rebase，但 Practical 这里重点理解 fetch 加 merge 的基本模型。

命令	下载远程 commit	修改当前文件	自动整合
git fetch	是	否	否
git pull	是	是	是

表 7: fetch 与 pull 的区别

什么时候用哪个？

- 个人仓库、修改简单：通常可以直接 git pull。
- 小组项目、希望先检查别人修改了什么：先 git fetch，检查后再 merge。

小组项目中较谨慎的流程是：

```
git fetch origin
git log HEAD..origin/main --oneline
git diff HEAD origin/main
git merge origin/main
```

9 Exercise 6: Reflection Log

最后需要记录本周遇到的问题 and 学到的内容，例如：

- SSH 配置过程中哪里出错？
- commit 和 push 有什么区别？
- 为什么 .gitignore 没有忽略已经 commit 的文件？
- 什么情况下应当使用 fetch 而不是 pull？

可以写成：

I initially confused committing with uploading to GitHub. I now understand that a commit is stored locally, while push transfers local commits to the remote repository. I would use git fetch before merging when working in a shared repository because it allows me to inspect collaborators' changes first.

然后提交 reflection：

```
git add reflection-log.qmd
git commit -m "add week 4 reflection log"
git push
```

10 小组项目中的推荐工作流

```
# 开始工作前
git status
git pull

# 编辑代码和文档

# 提交前检查
git status
git diff

# 明确添加本次要提交的文件
git add code/03_analysis.R

# 检查即将提交的修改
git diff --staged

# 创建本地版本并上传
git commit -m "add indexed age trend analysis"
git push
```

不要机械地使用 `git add .`

`git add .` 会暂存当前目录下所有未忽略的修改。在正式项目中，更稳妥的做法是明确添加本次需要提交的文件，并使用 `git status` 和 `git diff --staged` 检查即将提交的内容，避免把无关文件、临时文件或未完成的修改一起提交。

11 命令速查表

命令	作用
<code>git status</code>	查看当前仓库状态
<code>git init</code>	把当前文件夹初始化为 Git 仓库
<code>git add <file></code>	把指定修改加入暂存区
<code>git commit -m "..."</code>	在本地创建一个 commit
<code>git log --oneline</code>	简洁显示 commit 历史
<code>git show <hash></code>	查看指定 commit 的修改
<code>git remote -v</code>	查看远程仓库地址
<code>git remote add origin <URL></code>	添加名为 origin 的远程仓库

命令	作用
<code>git push</code>	上传本地 commit
<code>git pull</code>	下载并整合远程修改
<code>git fetch origin</code>	下载远程信息，但不修改当前文件
<code>git diff</code>	查看尚未暂存的修改
<code>git diff --staged</code>	查看即将进入下一次 commit 的修改
<code>git clone <URL></code>	克隆完整仓库及其历史
<code>git restore</code> <code>--source=<hash> <file></code>	从旧 commit 恢复文件
<code>git revert <hash></code>	用新 commit 撤销旧 commit
<code>git rm --cached <file></code>	停止跟踪文件，但保留本地副本

中文核心总结

1. 保存文件只改变 Working Directory。
2. `git add` 决定下一次 commit 包含什么。
3. `git commit` 在本地创建版本。
4. `git push` 才把版本上传到 GitHub。
5. `.gitignore` 只影响尚未被跟踪的文件。
6. `git fetch` 只下载和更新远程记录；`git pull` 还会立即整合。
7. 共享仓库中优先保留清晰历史，不要随意删除 `.git` 或改写公共历史。

第二部分 Deutsche Zusammenfassung

12 Lernziele

Das Practical behandelt einen vollständigen lokalen und entfernten Git-Workflow:

Working Directory $\xrightarrow{\text{git add}}$ **Staging Area** $\xrightarrow{\text{git commit}}$ **lokales Repository** $\xrightarrow{\text{git push}}$ **GitHub**

Die zentrale Unterscheidung lautet:

- `git add` wählt Änderungen für den nächsten Commit aus.
- `git commit` speichert einen neuen Versionsstand lokal.
- `git push` überträgt lokale Commits zum Remote-Repository.

13 Konfiguration und Diagnose

```
git config --global user.name
git config --global user.email
git status
git log --oneline
git remote -v
```

`git status` zeigt den aktuellen Branch sowie geänderte, gestagte und ungetrackte Dateien. `git log --oneline` zeigt die Commit-Historie in kompakter Form. `git remote -v` zeigt die konfigurierten Remote-Adressen.

Falls das Verzeichnis noch kein Repository ist:

```
git init
git add .
git commit -m "add debugging exercise"
```

Vorsicht mit `.git`

Das Löschen des Verzeichnisses `.git` entfernt die gesamte lokale Versionsgeschichte, Branches, Remotes und Stashes. Dies ist keine normale Reparaturmethode für echte Projekte.

14 Lokales Repository mit GitHub verbinden

```
git remote add origin git@github.com:USERNAME/REPOSITORY.git
git branch -M main
git push -u origin main
```

`origin` ist der übliche Name des Remote-Repositorys. Mit `-u` wird `origin/main` als Upstream des lokalen Branches `main` gespeichert. Danach genügen normalerweise `git push` und `git pull`.

Für ein bereits bestehendes lokales Repository sollte das neue GitHub-Repository zunächst leer sein. Ein automatisch erzeugtes README würde einen zusätzlichen Remote-Commit erzeugen und kann den ersten Push unnötig komplizieren.

15 .gitignore

`.gitignore` verhindert, dass bestimmte noch nicht getrackte Dateien in die Versionskontrolle aufgenommen werden:

```
.Rhistory
.RData
.Rproj.user/
/.quarto/
/_site/
.DS_Store
Thumbs.db
```

Bereits commitete Dateien werden durch einen späteren Eintrag in `.gitignore` nicht automatisch ungetrackt. Dafür ist beispielsweise nötig:

```
git rm --cached path/to/file
git commit -m "stop tracking file"
git push
```

`--cached` entfernt die Datei aus dem Git-Index, behält sie aber lokal.

16 Historie prüfen und Dateien wiederherstellen

```
git log --oneline --graph
git log -- practical3.qmd
git show <commit-hash>
git show <commit-hash>:practical3.qmd
```

Eine einzelne Datei kann aus einem älteren Commit wiederhergestellt werden:

```
git restore --source=<commit-hash> --staged --worktree practical3.qmd
```

Alternativ funktioniert die ältere Schreibweise:

```
git checkout <commit-hash> -- practical3.qmd
```

`git revert <hash>` hat eine andere Funktion: Es erstellt einen neuen Commit, der die Änderungen eines älteren Commits rückgängig macht. In geteilten Repositories ist dies häufig sicherer als das Umschreiben der Historie.

17 Repository klonen und synchronisieren

```
git clone git@github.com:USERNAME/REPOSITORY.git statprog-copy
cd statprog-copy
git add .
git commit -m "add comment from practical 4"
git push
```

`git clone` lädt nicht nur Dateien herunter, sondern kopiert auch die Commit-Historie, richtet `origin` ein und checkt den Standard-Branch aus.

Vor einem Pull sollte der lokale Zustand geprüft werden:

```
git status
git pull
```

Lokale, nicht commitete Änderungen können einen Pull blockieren oder zu Merge-Konflikten führen.

18 fetch und pull

Befehl	Remote-Commits laden	Arbeitsdateien ändern	Automatisch integrieren
<code>git fetch</code>	Ja	Nein	Nein
<code>git pull</code>	Ja	Ja	Ja

表 9: Unterschied zwischen `fetch` und `pull`

`git fetch origin` aktualisiert die Remote-Tracking-Information `origin/main`, ohne den lokalen Branch oder die Arbeitsdateien zu verändern:

```
git fetch origin
git log HEAD..origin/main --oneline
git diff HEAD origin/main
git merge origin/main
```

`git pull` kann im Grundmodell als `fetch` plus anschließende Integration verstanden werden. Je nach Konfiguration kann diese Integration über Merge oder Rebase erfolgen.

19 Empfohlener Workflow für das Gruppenprojekt

```
git status
git pull
```

```
# Dateien bearbeiten

git status
git diff
git add code/03_analysis.R
git diff --staged
git commit -m "add indexed age trend analysis"
git push
```

Es ist sinnvoll, konkrete Dateien zu stagen, statt automatisch immer `git add .` zu verwenden. So lässt sich vor dem Commit besser kontrollieren, welche Änderungen tatsächlich zur neuen Version gehören.

Deutsche Kurzfassung

1. Ein Commit ist ein lokaler Versionsstand; ein Push lädt ihn zu GitHub hoch.
2. `.gitignore` wirkt grundsätzlich nur auf noch nicht getrackte Dateien.
3. `git fetch` lädt Remote-Informationen ohne automatische Integration.
4. `git pull` lädt Remote-Änderungen und integriert sie sofort.
5. Vor `pull`, `commit` und `push` sollte `git status` geprüft werden.
6. In geteilten Repositories sollte die Historie nachvollziehbar bleiben.

Quelle

- LMU Statistical Programming II, Practical 4: <https://soda-lmu.github.io/StatProg2-2026-SoSe/04-version-control/practical.html>